

Pontificia Universidad Católica Madre y Maestra

Facultad de Ciencias e Ingeniería
Escuela de Ingeniería en Computación y Telecomunicaciones

ICC-321-T Inteligencia de Negocios, Grupo 5227

Práctica 1

Diseño e implementación de una solución de Inteligencia de Negocios

Presentado por:

Elías De La Cruz Jiménez, 1015-5971
Enger Douglas, 1014-4675

Presentado a:

Prof. Lisibonny Beato

Santiago de los Caballeros, República Dominicana
27 de septiembre de 2026

Índice

1. Introducción.....	3
2. Descripción general de la fuente de datos.....	3
2.1 Entidades y relaciones.....	4
2.2 Naturaleza y granularidad de los datos.....	5
2.3 Variables disponibles para el análisis.....	5
2.4 Problemas de calidad de datos.....	6
2.5 Posibilidades de análisis.....	6
3. Problema de toma de decisiones.....	6
3.1 Situación encontrada en los datos.....	6
3.2 Problema y necesidad de información.....	7
3.3 Usuarios de la solución.....	7
3.4 Decisiones que se busca apoyar.....	7
3.5 Preguntas de análisis.....	8
4. Diseño del modelo dimensional.....	8
4.1 Proceso de negocio y grano.....	8
4.2 Medidas.....	8
4.3 Dimensiones.....	9
4.4 Esquema estrella.....	9
4.5 Claves y relaciones.....	10
4.6 Relación entre preguntas, medidas y dimensiones.....	11
5. Implementación del Data Warehouse.....	11
6. Proceso ETL en Tableau Prep.....	13
7. Análisis de las medidas.....	15
7.1 Medidas de FactVentas.....	15
7.2 Medidas de FactDevoluciones.....	16
7.3 Medida de FactPagos.....	17
7.4 Indicadores calculados en el dashboard.....	17
8. Dashboard interactivo.....	17
8.1 Tablero 1: rentabilidad de la venta.....	18
8.2 Tablero 2: devoluciones y proveedores.....	19
8.3 Relación entre las preguntas y el dashboard.....	21
9. Justificación de las principales decisiones de diseño.....	21
10. Conclusiones.....	22
Referencias.....	22
Anexo A. Script de creación del Data Warehouse.....	24
Anexo B. Script de carga del Data Warehouse.....	28
Anexo C. Detalle del flujo ETL en Tableau Prep.....	31
Anexo D. Consultas y campos calculados del dashboard.....	35

1. Introducción

En esta práctica diseñamos e implementamos una solución de Inteligencia de Negocios completa a partir de la base de datos operacional BI_Practica_Retail, que se crea con el script entregado junto al enunciado. La base registra las ventas de una cadena de tiendas al detalle que vende en 12 sucursales físicas y por dos canales digitales (Web y App), entre el 1 de enero de 2023 y el 15 de agosto de 2026.

El trabajo siguió el orden que plantea el enunciado. Primero exploramos la fuente para entender sus tablas, su granularidad y sus problemas de calidad. Con eso definimos un problema de decisión y las preguntas de análisis que debía responder la solución. Luego diseñamos un modelo dimensional con la metodología de Kimball [1], lo implementamos como Data Warehouse en SQL Server, construimos el proceso ETL en Tableau Prep y, por último, desarrollamos un dashboard interactivo en Tableau Desktop que lee únicamente del Data Warehouse. Es el recorrido habitual de una solución de BI, que va de la integración de los datos a su análisis [2]. La Figura 1 lo resume.



Figura 1. Arquitectura de la solución, desde la base operacional hasta el dashboard.

Este documento incluye todos los elementos de la solución. El script completo que crea el Data Warehouse está en el Anexo A y el script de carga en el Anexo B. El detalle del flujo de Tableau Prep, paso por paso y con cada cálculo, está en el Anexo C, y las consultas y los campos calculados del dashboard están en el Anexo D. Las secciones 6 y 8 muestran el flujo y el dashboard tal como se ven en Tableau.

2. Descripción general de la fuente de datos

Para crear la base ejecutamos el script de la práctica en SQL Server 2022. El script original no corrió en el primer intento: en el bloque que genera los detalles de orden usa el alias de columna LineNo, y LINENO es una palabra reservada de Transact-SQL, por lo que el motor devuelve el error "Incorrect syntax near the keyword LineNo". Renombramos ese alias a LineaNo en sus 9

apariciones y con ese cambio el script corrió completo. Guardamos esa versión como Retail_Operacional_fix.sql.

Además notamos que el script no genera siempre los mismos datos. Inserta las filas con claves IDENTITY desde consultas sin ORDER BY y a partir de esas claves calcula fechas, productos y devoluciones, así que el resultado depende del orden en que el motor asigne las claves. Cuando lo ejecutamos en las computadoras de los dos integrantes obtuvimos, por ejemplo, 5,139 devoluciones en una y 5,181 en la otra. Para que todas las etapas trabajaran sobre los mismos datos, extrajimos las 10 tablas de una sola ejecución a archivos CSV y todo el trabajo posterior se hizo sobre esa extracción. Las cifras de este documento corresponden a ella.

2.1 Entidades y relaciones

La base tiene 10 tablas en el esquema dbo, relacionadas por 9 claves foráneas. Seis son tablas de catálogo, que describen el negocio, y cuatro son transaccionales, que registran lo que ocurre en la operación (Tabla 1).

Tabla 1. Tablas de la base operacional BI_Practica_Retail.

Tabla	Tipo	Filas	Contenido
Categoria	Catálogo	10	Categorías comerciales de producto
Proveedor	Catálogo	24	Proveedores, con país y fecha de alta
Producto	Catálogo	250	SKU, nombre, marca, costo unitario, precio de lista y si está activo
Tienda	Catálogo	12	Sucursales, con ciudad, provincia, tipo y fecha de apertura
Cliente	Catálogo	2,500	Datos demográficos, ubicación, segmento y fecha de registro
Promocion	Catálogo	15	Campañas con vigencia, porcentaje de descuento y canal
OrdenVenta	Transaccional	30,000	Cabecera de la venta: fecha y hora, tienda, cliente, canal, estado y promoción
DetalleOrden	Transaccional	76,784	Líneas de la venta: producto, cantidad, precio unitario y descuento
Pago	Transaccional	28,803	Pago de la orden: fecha, método y monto
Devolucion	Transaccional	5,139	Devolución de una línea: fecha, cantidad, motivo y reembolso

Una orden pertenece a una tienda y, de forma opcional, a un cliente y a una promoción. Cada orden tiene entre 1 y 4 líneas de detalle y cada línea corresponde a un producto, que a su vez

pertenece a una categoría y a un proveedor. El pago se registra por orden completa, mientras que la devolución se registra sobre una línea de detalle.

2.2 Naturaleza y granularidad de los datos

Revisamos qué representa una fila de cada tabla transaccional, porque de eso depende el diseño del modelo. OrdenVenta tiene una fila por venta (30,000 filas y 30,000 identificadores distintos). DetalleOrden tiene una fila por producto dentro de una orden: 76,784 líneas, con un promedio de 2.56 líneas por orden; 6,099 órdenes tienen una línea, 8,009 tienen dos, 8,901 tienen tres y 6,991 tienen cuatro. Pago tiene exactamente un pago por orden completada (28,803 pagos para 28,803 órdenes) y Devolucion tiene una fila por línea devuelta (5,139 filas con 5,139 líneas distintas).

La conclusión práctica es que el pago y la devolución no están al mismo grano que la línea de venta. Si el monto del pago se copiara en cada línea, se repetiría tantas veces como líneas tenga la orden y cualquier suma quedaría inflada. Por eso los pagos y las devoluciones se trataron como hechos aparte.

2.3 Variables disponibles para el análisis

Las medidas numéricas están en DetalleOrden (cantidad, precio unitario y descuento), en Producto (costo unitario y precio de lista), en Pago (monto) y en Devolucion (cantidad devuelta y monto reembolsado). Para analizar esas medidas hay atributos de tiempo (fecha y hora de la orden, fecha de pago, fecha de devolución), de producto (SKU, nombre, 12 marcas, 10 categorías y 24 proveedores con su país), de cliente (sexo, fecha de nacimiento, ciudad, 8 provincias y segmento), de tienda (ciudad, provincia y tipo) y de la venta (canal, estado, método de pago, promoción y motivo de devolución).

Tabla 2. Distribuciones observadas en la fuente.

Variable	Distribución
Canal de venta	Tienda 15,223 órdenes (50.7%), Web 8,252 (27.5%) y App 6,525 (21.8%)
Estado de la orden	Completada 28,803 (96.01%) y Cancelada 1,197 (3.99%)
Segmento de cliente	Regular 1,553, Preferente 643 y Corporativo 304 clientes
Tipo de tienda	Calle 4, Centro comercial 4, Express 3 y Outlet 1
Promoción	5,966 órdenes (19.9%) tienen promoción, con descuentos entre 10% y 20%
Método de pago	Cinco métodos, con entre 5,755 y 5,766 pagos cada uno

2.4 Problemas de calidad de datos

Durante la exploración encontramos varias situaciones que había que resolver en el ETL. Ninguna obligó a descartar datos, salvo las órdenes canceladas (Tabla 3).

Tabla 3. Problemas de calidad encontrados y tratamiento aplicado.

Problema	Magnitud	Tratamiento
Órdenes sin cliente (ClienteID nulo)	2,407 órdenes (8.02%)	Se asignan a un miembro "No identificado" con clave -1 en la dimensión Cliente
Cientes sin sexo registrado	254 de 2,500 (10.2%)	El nulo se reemplaza por "No especificado"
Órdenes canceladas con líneas de detalle	1,197 órdenes, 3,117 líneas, RD\$12.1 millones	Se excluyen del hecho de ventas, porque no son ventas efectivas
Órdenes sin promoción	24,034 órdenes (80.1%)	Se asignan a un miembro "Sin promoción" con clave -1
Ventas de productos marcados como inactivos	3,123 líneas de 10 productos	Se conservan, porque son ventas históricas válidas

También verificamos que no hay devoluciones con más unidades que las vendidas, ni devoluciones o pagos de órdenes canceladas, que el monto de cada pago coincide con el neto de su orden y que no hay claves foráneas huérfanas.

2.5 Posibilidades de análisis

Con estos datos se puede analizar la rentabilidad por categoría, marca y proveedor, el efecto de las promociones, las devoluciones y su costo, el desempeño por canal y tipo de tienda, el comportamiento de los segmentos de clientes y la evolución de la venta en el tiempo. Como referencia, las órdenes completadas suman 210,229 unidades, una venta neta de RD\$288,991,557.74 y un margen bruto de RD\$75,608,585.04, es decir, 26.16% sobre la venta neta.

3. Problema de toma de decisiones

3.1 Situación encontrada en los datos

A nivel general el negocio se ve estable: vende entre RD\$77 y 82 millones al año y su margen se mantiene cerca de 26% en todos los años. Sin embargo, cuando ese margen se separa por promoción, por categoría y por devoluciones aparecen tres problemas que un reporte de ingresos no deja ver.

El primero es que las promociones reducen mucho el margen sin generar más venta por orden. Las órdenes con promoción dejan un margen de 17.09%, frente a 28.13% de las que no tienen promoción, y su ticket promedio es menor (RD\$8,965 contra RD\$10,299). Las unidades por orden son prácticamente las mismas (7.29 contra 7.30). El segundo es que la categoría que más vende no es la que más aporta: Papelería lidera la venta neta con RD\$38.7 millones, pero tiene el margen más bajo (23.3%), mientras que Cuidado personal, con RD\$29.2 millones, deja 27.9%. El tercero es que las devoluciones se concentran en pocas categorías: Moda devuelve el 14.0% de sus líneas y Electrodomésticos el 10.0%, cuando el promedio es 6.98%. En total se reembolsaron RD\$13.2 millones, lo que equivale al 17.4% del margen bruto.

3.2 Problema y necesidad de información

La empresa mide su desempeño con información de ingresos, pero no puede medir la rentabilidad real de sus categorías y productos, porque los descuentos y las devoluciones se registran en procesos separados de la venta. Por eso puede estar poniendo en promoción categorías cuyo margen no aguanta el descuento y manteniendo productos con muchas devoluciones sin saber cuánto le cuestan. Lo que hace falta es una vista integrada donde se puedan seguir juntos la venta, el descuento, el costo, el margen y los reembolsos, y separarlos por categoría, producto, promoción, canal, tienda, cliente y tiempo.

3.3 Usuarios de la solución

El usuario principal es el gerente comercial o de categorías, que decide el surtido, los precios y qué categorías entran en promoción. También la usarían el gerente de mercadeo, para evaluar las campañas; la gerencia de tiendas y canales, para comparar sucursales y el canal físico con el digital; y la gerencia general, para seguir el margen consolidado.

3.4 Decisiones que se busca apoyar

- Qué categorías, marcas y productos deben entrar o salir de las campañas promocionales.
- Qué nivel de descuento puede soportar cada categoría sin perder su margen.
- Sobre qué productos actuar para reducir las devoluciones.
- Qué proveedores conviene renegociar, según el margen y las devoluciones que generan.
- Cómo repartir el esfuerzo comercial entre tiendas y canales.

3.5 Preguntas de análisis

A partir de lo anterior definimos ocho preguntas, que son las que guían el modelo y el dashboard (Tabla 4).

Tabla 4. Preguntas de análisis.

#	Pregunta
P1	¿Qué categorías, marcas y productos aportan más margen bruto, y no solo más ingresos?
P2	¿Cuánto margen sacrifican las promociones y qué volumen adicional generan realmente?
P3	¿Qué productos y categorías concentran las devoluciones y cuánto cuestan en reembolsos?
P4	¿Cuál es el margen neto de devoluciones por categoría y por proveedor?
P5	¿Cómo evolucionan la venta y el margen mes a mes, y qué meses concentran la demanda?
P6	¿Qué canal de venta y qué tipo de tienda rinden mejor en ticket promedio y margen?
P7	¿Qué segmentos de cliente sostienen la venta y qué peso tienen las ventas sin cliente identificado?
P8	¿Qué proveedores concentran el margen y cuáles arrastran las devoluciones?

4. Diseño del modelo dimensional

4.1 Proceso de negocio y grano

El proceso de negocio que modelamos es la venta al detalle de productos en tiendas y canales digitales. Siguiendo a Kimball y Ross [1], el grano se declara antes de elegir medidas y dimensiones. El grano de la tabla de hechos principal es una línea de detalle de una orden de venta completada, es decir, una fila por cada producto vendido dentro de una orden. Elegimos este grano porque es el nivel más fino que ofrece la fuente y permite resumir hacia arriba (por orden, producto, categoría, tienda, día o mes) sin perder información. Las 1,197 órdenes canceladas y sus 3,117 líneas quedan fuera porque no representan ventas.

4.2 Medidas

Las medidas de FactVentas se obtienen de la línea de detalle y del costo del producto (Tabla 5). Seis de ellas son aditivas y se pueden sumar por cualquier dimensión. El precio unitario y el margen porcentual no son aditivos: el primero se promedia y el segundo se calcula en el dashboard a partir de sus componentes.

Tabla 5. Medidas de FactVentas.

Medida	Cómo se obtiene	Aditividad
CantidadVendida	DetalleOrden.Cantidad	Aditiva
MontoBruto	Cantidad × PrecioUnitario	Aditiva
DescuentoMonto	DetalleOrden.DescuentoMonto	Aditiva
MontoNeto	MontoBruto - DescuentoMonto	Aditiva
CostoTotal	Cantidad × Producto.CostoUnitario	Aditiva
MargenBruto	MontoNeto - CostoTotal	Aditiva
PrecioUnitario	DetalleOrden.PrecioUnitario	No aditiva (se promedia)
MargenPorcentual	MargenBruto / MontoNeto	No aditiva (no se almacena)

4.3 Dimensiones

Tabla 6. Dimensiones del modelo.

Dimensión	Atributos principales	Observaciones
DimFecha	Fecha, año, trimestre, mes, semana, día de la semana, fin de semana	Cubre del 1-1-2023 al 31-12-2026. Se usa como fecha de venta, de pago y de devolución
DimProducto	SKU, nombre, marca, categoría, proveedor, país del proveedor, rango de precio, activo	Categoría y proveedor se incluyen dentro de la dimensión (esquema estrella, no copo de nieve)
DimCliente	Nombre, sexo, grupo de edad, ciudad, provincia, segmento, año de registro	Incluye el miembro "No identificado" con clave -1
DimTienda	Nombre, ciudad, provincia, tipo de tienda, año de apertura	12 sucursales
DimPromocion	Nombre, porcentaje de descuento, canal, vigencia	Incluye el miembro "Sin promoción" con clave -1
DimCanal	Canal de venta	Tienda, Web y App
DimMotivoDevolucion	Motivo	Cinco motivos. Solo aplica a las devoluciones
DimMetodoPago	Método de pago	Cinco métodos. Solo aplica a los pagos

4.4 Esquema estrella

La Figura 2 muestra el esquema estrella de FactVentas con sus seis dimensiones. OrdenID se guarda en el hecho como dimensión degenerada, porque identifica la orden pero no tiene atributos propios que justifiquen una tabla aparte.

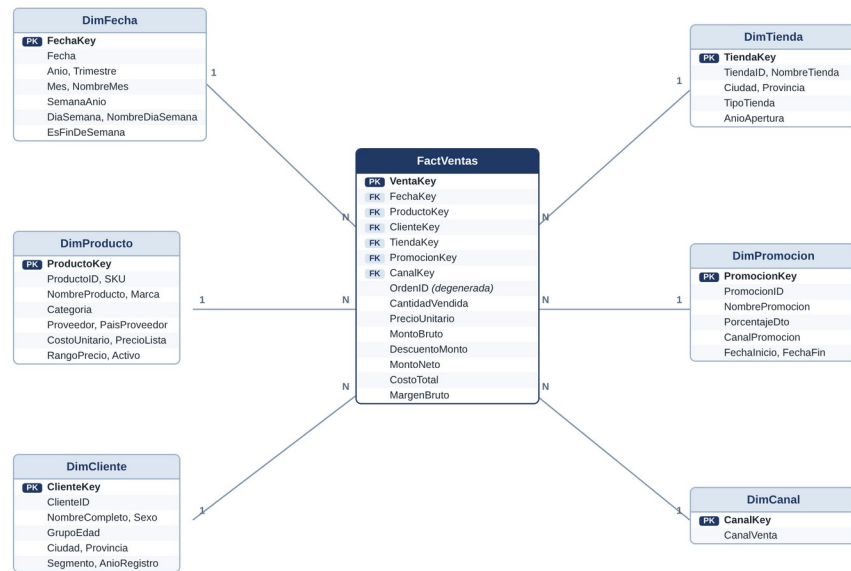


Figura 2. Esquema estrella de FactVentas.

El modelo se completa con dos hechos más, que no se pueden meter en FactVentas porque están a otro grano: FactDevoluciones, con una fila por devolución de una línea, y FactPagos, con una fila por pago de una orden. Los tres hechos comparten las dimensiones de fecha, cliente y tienda, lo que permite filtrarlos de la misma forma. La Tabla 7 es la matriz de bus del modelo.

Tabla 7. Matriz de bus.

Hecho	Fecha	Producto	Cliente	Tienda	Promoción	Canal	Motivo	Método de pago
FactVentas	X	X	X	X	X	X		
FactDevoluciones	X	X	X	X			X	
FactPagos	X		X	X		X		X

4.5 Claves y relaciones

Cada dimensión tiene una clave subrogada entera, propia del Data Warehouse. En este caso la asignamos a partir del identificador del origen y además guardamos la clave natural como atributo, para poder rastrear de dónde viene cada fila. DimFecha usa como clave un entero con formato AAAAMMDD, que se lee fácil y es la práctica que recomienda Kimball [1]. Los hechos guardan solo las claves foráneas, las medidas y la dimensión degenerada, y todas las relaciones son de uno a muchos desde la dimensión hacia el hecho. Los miembros con clave -1 evitan que queden hechos sin relación cuando el origen trae nulos.

La Figura 3 muestra los esquemas de los otros dos hechos. FactDevoluciones comparte con FactVentas las dimensiones de fecha, producto, cliente y tienda, y agrega el motivo de la devolución. FactPagos comparte fecha, cliente, tienda y canal, y agrega el método de pago. En los dos casos DimFecha representa una fecha distinta a la de la venta: la de la devolución y la del pago.

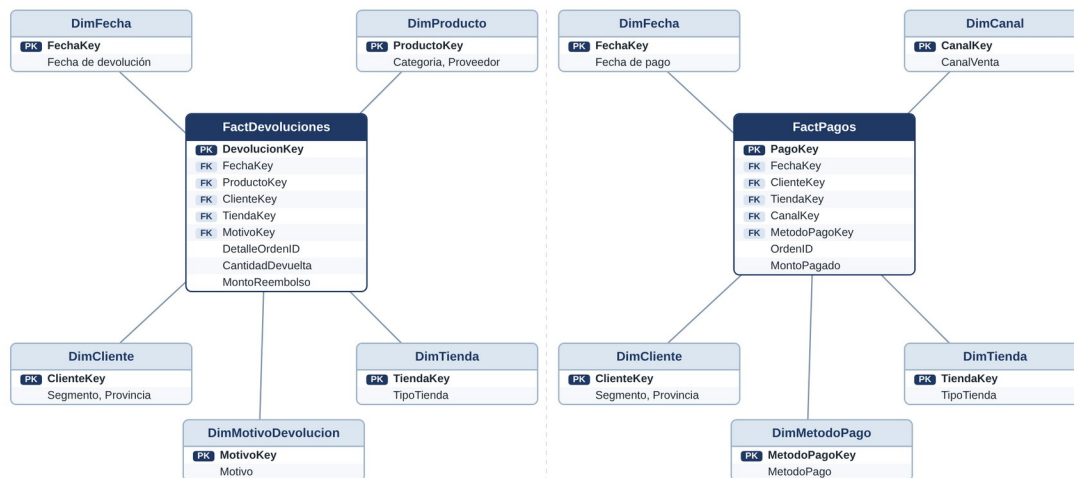


Figura 3. Esquemas de FactDevoluciones (izquierda) y FactPagos (derecha).

4.6 Relación entre preguntas, medidas y dimensiones

Tabla 8. Trazabilidad entre preguntas, medidas y dimensiones.

Pregunta	Medidas	Dimensiones
P1	MargenBruto, MontoNeto, margen %	Producto (categoría, marca), Fecha
P2	MontoNeto, DescuentoMonto, MargenBruto, CantidadVendida	Promoción, Producto, Fecha
P3	CantidadDevuelta, MontoReembolso	Producto, Motivo, Fecha
P4	MargenBruto - MontoReembolso	Producto (categoría, proveedor)
P5	MontoNeto, MargenBruto	Fecha (año, trimestre, mes)
P6	MontoNeto, MargenBruto, ticket promedio	Canal, Tienda (tipo de tienda)
P7	MontoNeto, CantidadVendida	Cliente (segmento, provincia)
P8	MargenBruto, MontoReembolso	Producto (proveedor, país)

5. Implementación del Data Warehouse

El modelo se implementó en SQL Server con el script DW_Retail.sql (Anexo A), que crea la base DW_Retail desde cero. Las ocho dimensiones tienen su clave subrogada como PRIMARY

KEY y los tres hechos tienen una clave propia de tipo bigint tomada del identificador de la línea, la devolución o el pago del origen. En total hay 16 claves foráneas (6 en FactVentas, 5 en FactDevoluciones y 5 en FactPagos), de manera que ningún hecho puede apuntar a un miembro que no exista.

Para los montos usamos decimal(14,2) y para los precios decimal(12,2), así no se pierden centavos. Todas las columnas de los hechos son NOT NULL, y las cantidades vendidas y devueltas tienen un CHECK que exige que sean mayores que cero. También creamos 8 índices sobre las claves foráneas que más se usan en las consultas (fecha, producto, cliente, tienda y promoción).

DimFecha se llena en el mismo script con los 1,461 días del 1 de enero de 2023 al 31 de diciembre de 2026, sin depender de los hechos. Antes de llenarla fijamos SET DATEFIRST 7, porque el número del día de la semana y de la semana del año dependen de esa configuración del servidor [4]. El script también inserta los miembros "No identificado" y "Sin promoción" con clave -1.

La carga la hace Cargar_DW.sql (Anexo B), que lee los CSV que produce el flujo de Tableau Prep con BULK INSERT [3]. Como Tableau Prep escribe las columnas en orden alfabético, los archivos se cargan primero en tablas temporales y de ahí se insertan en las definitivas con el tipo de dato correcto. Durante esta parte tuvimos que resolver algunos problemas que dependían de la computadora donde se ejecutaba:

- En SQL Server para Linux la opción CODEPAGE de BULK INSERT no está disponible, así que la quitamos después de comprobar que los datos no tienen caracteres fuera de ASCII.
- Tableau Prep escribe las fechas como mes/día/año. Para que se lean igual en cualquier servidor, el script fija SET DATEFORMAT mdy.
- Tableau Prep en Windows termina las líneas de los CSV con CRLF y en macOS con LF. Después de cada carga se elimina el retorno de carro que queda en la última columna, y así el script funciona con archivos de cualquiera de las dos computadoras.
- La carpeta de los CSV se indica en la variable RutaCSV al principio del script, que se ejecuta en modo SQLCMD.

Después de la carga comparamos el Data Warehouse con la fuente. Los conteos y los totales coinciden (Tabla 9). La prueba más importante es la de la venta neta: FactVentas y FactPagos

suman exactamente RD\$288,991,557.74, lo mismo que las órdenes completadas del origen, lo que confirma que no se perdió ni se duplicó ninguna línea al pasar de un grano al otro.

Tabla 9. Comprobación de la carga del Data Warehouse.

Tabla	Filas	Comprobación
FactVentas	73,667	76,784 líneas menos las 3,117 de órdenes canceladas
FactDevoluciones	5,139	Todas las devoluciones del origen
FactPagos	28,803	Un pago por cada orden completada
DimCliente	2,501	2,500 clientes más el miembro "No identificado"
DimPromocion	16	15 promociones más el miembro "Sin promoción"
DimProducto / DimTienda	250 / 12	Igual al catálogo de origen
DimCanal / DimMotivoDevolucion / DimMetodoPago	3 / 5 / 5	Valores distintos del origen
DimFecha	1,461	Todos los días de 2023 a 2026

6. Proceso ETL en Tableau Prep

El ETL se construyó en Tableau Prep Builder 2026.2 (Figura 4). El flujo tiene 40 nodos: 10 entradas, una por cada tabla extraída del origen; 11 pasos de limpieza; 6 uniones; 3 agregaciones y 10 salidas, una por cada tabla del Data Warehouse. Las salidas se escriben como CSV en la carpeta CSV_salida y de ahí las carga Cargar_DW.sql. Guardamos el flujo también como flujo empaquetado (.tflx), que lleva los archivos de entrada dentro y se puede abrir en otra computadora sin cambiar rutas [7]. En el Anexo C están todos los pasos con sus cálculos.

El flujo se puede leer en tres bloques. Arriba están las dimensiones de catálogo: DimProducto une Producto con Categoría y con Proveedor, y DimCliente, DimTienda y DimPromocion limpian cada tabla por separado. En el medio están las dimensiones que salen de los datos transaccionales: DimCanal, DimMotivoDevolucion y DimMetodoPago se obtienen agregando los valores distintos de las órdenes, las devoluciones y los pagos. Abajo están los hechos, que parten de las órdenes completadas y se unen con el detalle, el producto, las devoluciones y los pagos [6]. La Tabla 10 resume las transformaciones y la razón de cada una.

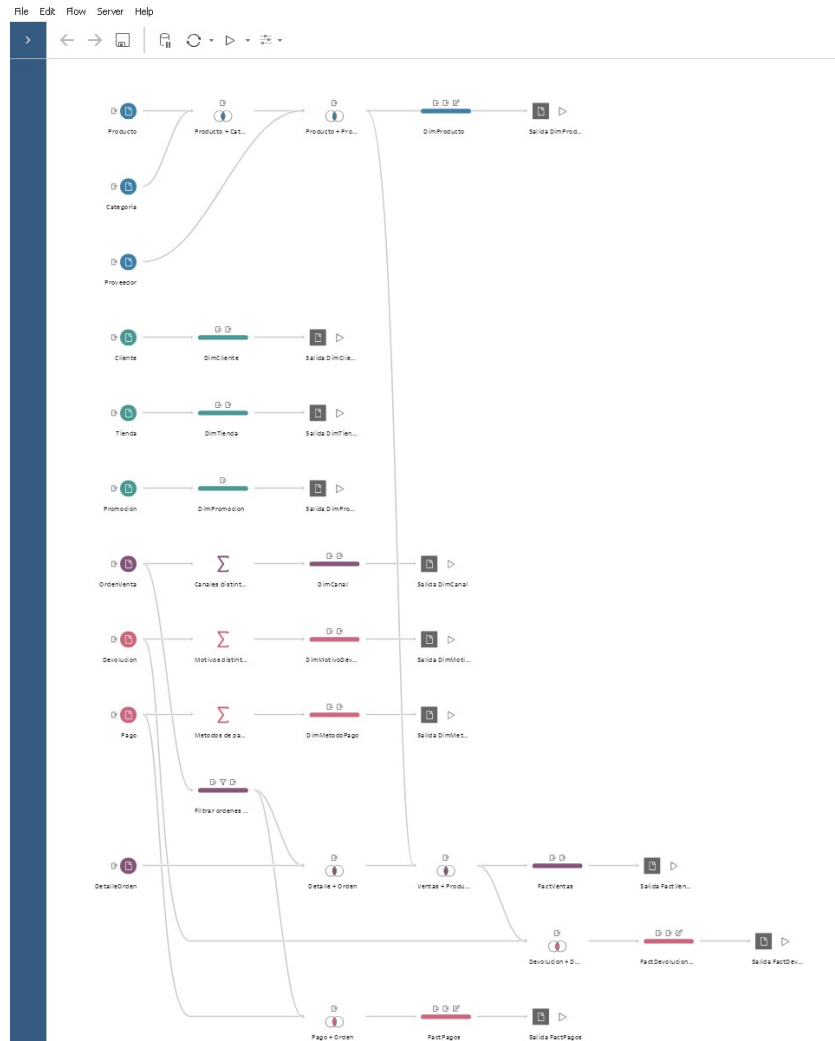


Figura 4. Flujo ETL_DW_Retail en Tableau Prep Builder.

Tabla 10. Transformaciones del flujo ETL.

Paso	Transformación	Motivo
Producto + Categoría + Proveedor	Dos uniones internas por CategoríaID y ProveedorID	DimProducto lleva la categoría y el proveedor dentro (esquema estrella)
DimProducto	Clave subrogada y rango de precio (Económico hasta RD\$800, Medio hasta RD\$1,500, Premium)	Permite agrupar el margen por rango de precio
DimCliente	Une nombre y apellido, reemplaza el sexo nulo, calcula la edad al 15 de agosto de 2026 y el grupo de edad, y el año de registro	Atributos para analizar a los clientes (P7). La fecha fija hace que el grupo no cambie según el día en que se corre el flujo
DimTienda, DimPromoción	Clave subrogada y año de apertura	Dimensiones de P2 y P6

Paso	Transformación	Motivo
DimCanal, DimMotivoDevolucion, DimMetodoPago	Agregación por valor distinto y clave fija para cada valor	En el origen son solo texto dentro de las transacciones
Filtrar órdenes completadas	Filtro EstadoOrden = "Completada", clave de fecha AAAAMMDD, cliente y promoción nulos como -1, clave de canal	Grano del hecho y miembros -1
Detalle + Orden + Producto	Une cada línea con su orden y con el costo unitario del producto	El costo está en el catálogo, no en la línea
FactVentas	Calcula MontoBruto, MontoNeto, CostoTotal y MargenBruto redondeados a dos decimales	Las medidas quedan listas para sumarse
FactDevoluciones	Une la devolución con su línea, clave de fecha de la devolución y clave del motivo	Hecho aparte, al grano de la devolución
FactPagos	Une el pago con su orden, clave de fecha del pago y clave del método	Hecho al grano de la orden

Un detalle que nos tomó tiempo fue el del redondeo. Al principio Tableau Prep escribía algunos márgenes en notación científica (por ejemplo 2.27e-13) cuando el resultado era prácticamente cero, y SQL Server no podía convertir ese texto. Lo resolvimos aplicando ROUND(..., 2) a las medidas monetarias dentro del flujo, que es además lo correcto para importes.

Para validar el ETL comparamos las salidas con el origen línea por línea. Los 73,667 registros de FactVentas tienen la misma venta neta, el mismo cliente, la misma promoción y la misma fecha que su línea de origen, y ningún registro quedó sin clave de canal, motivo o método de pago.

7. Análisis de las medidas

En esta sección explicamos cada medida: qué representa, cómo se obtiene, su relación con el grano, por qué dimensiones se puede analizar, qué preguntas responde y cómo se puede agregar. Los totales son de todo el período cargado.

7.1 Medidas de FactVentas

Todas las medidas de FactVentas están al grano de la línea de orden completada: cada fila las aporta una sola vez, así que cualquier resumen hacia arriba es una suma directa. Se pueden analizar por las seis dimensiones del hecho (fecha, producto con su categoría y proveedor, cliente, tienda, promoción y canal) y responden las preguntas P1, P2, P5, P6 y P7.

Tabla 11. Medidas de FactVentas con su total del período.

Medida	Qué representa y cómo se obtiene	Total	Agregación
CantidadVendida	Unidades del producto en la línea (DetalleOrden.Cantidad)	210,229 unidades	Aditiva
MontoBruto	Valor de la línea antes del descuento: $\text{Cantidad} \times \text{PrecioUnitario}$	RD\$298,749,966.36	Aditiva
DescuentoMonto	Descuento aplicado a la línea (DetalleOrden.DescuentoMonto)	RD\$9,758,408.62	Aditiva
MontoNeto	Lo que realmente se cobra: $\text{MontoBruto} - \text{DescuentoMonto}$	RD\$288,991,557.74	Aditiva
CostoTotal	Costo de la mercancía: $\text{Cantidad} \times \text{CostoUnitario}$ del producto	RD\$213,382,972.70	Aditiva
MargenBruto	Ganancia antes de devoluciones: $\text{MontoNeto} - \text{CostoTotal}$	RD\$75,608,585.04	Aditiva
PrecioUnitario	Precio cobrado por unidad en la línea	Promedio simple RD\$1,421.91	No aditiva

El precio unitario no se puede sumar. Además, su promedio simple (RD\$1,421.91) no es igual al precio promedio real ponderado por unidades, que es $\text{MontoBruto} / \text{CantidadVendida}$ (RD\$1,421.07). Por eso en el dashboard los precios y los porcentajes se calculan a partir de las medidas aditivas y no se promedian valores ya calculados.

También hay que tener en cuenta que DescuentoMonto no viene solo de las promociones. Las ventas sin promoción tienen algunos descuentos puntuales del 5%: son 7,159 líneas que suman RD\$1.5 millones, el 0.63% de su venta bruta. Es poco, por lo que el grupo "Sin promoción" sigue sirviendo como punto de comparación en la pregunta P2.

7.2 Medidas de FactDevoluciones

CantidadDevuelta (9,543 unidades en total) y MontoReembolso (RD\$13,158,252.77) se copian de la tabla Devolucion y están al grano de la devolución de una línea. Ambas son aditivas por fecha, producto, cliente, tienda y motivo, y responden P3, P4 y P8. La fecha de este hecho es la de la devolución, no la de la venta, por eso hay devoluciones hasta el 3 de septiembre de 2026 aunque la última venta es del 15 de agosto. Estas medidas no se pueden cruzar con promoción ni con canal, porque ese hecho no tiene esas dimensiones.

7.3 Medida de FactPagos

MontoPagado (RD\$288,991,557.74) está al grano de la orden, porque en el negocio se paga la orden completa y no cada línea. Es aditiva por fecha, cliente, tienda, canal y método de pago, pero no se puede repartir por producto ni por categoría. Su total es igual a la venta neta, lo que nos sirvió como control de la carga. La fecha es la del pago: en 1,727 de las 28,803 órdenes el pago ocurrió otro día, y en 56 de ellas en otro mes, así que al comparar por día o por mes los totales de FactPagos y FactVentas pueden no coincidir aunque el total del período sí coincida.

7.4 Indicadores calculados en el dashboard

Los indicadores que son razones no se guardan en el Data Warehouse; se calculan en Tableau con sus componentes, al nivel de detalle que se esté viendo (Tabla 12).

Tabla 12. Indicadores calculados a partir de las medidas.

Indicador	Fórmula	Valor del período	Preguntas
Margen %	$\text{SUM}(\text{MargenBruto}) / \text{SUM}(\text{MontoNeto})$	26.16%	P1, P2, P6, P8
Órdenes	$\text{COUNTD}(\text{OrdenID})$	28,803	P6
Ticket promedio	$\text{SUM}(\text{MontoNeto}) / \text{COUNTD}(\text{OrdenID})$	RD\$10,033	P2, P6
Tasa de devolución	Devoluciones / líneas vendidas	6.98%	P3, P8
Margen después de devoluciones	$\text{SUM}(\text{MargenBruto}) - \text{SUM}(\text{MontoReembolso})$	RD\$62,450,332.27	P4
Margen que se va en reembolsos	$\text{SUM}(\text{MontoReembolso}) / \text{SUM}(\text{MargenBruto})$	17.40%	P4, P8

Las órdenes se cuentan con COUNTD y nunca se suman entre grupos, porque una misma orden tiene varias líneas. Para el margen después de devoluciones no se puede unir FactVentas con FactDevoluciones fila por fila, porque las líneas con devolución se duplicarían. Lo que hicimos fue resumir cada hecho por separado al mismo nivel (año y producto) y luego combinarlos por esas dimensiones compartidas, que es lo que Kimball llama consulta entre hechos o drill-across [1].

8. Dashboard interactivo

El dashboard se hizo en Tableau Desktop y se guardó como libro empaquetado (Dashboard_Retail.twbx). Sus datos salen únicamente de DW_Retail: las consultas de Consultas_Dashboard.sql unen cada hecho con sus dimensiones y el resultado se guardó como

un extracto de Tableau que va dentro del libro [5]. Así el libro se puede abrir en cualquier computadora con Tableau sin conectarse al servidor, y ningún dato viene de la base operacional. Las consultas y los campos calculados están en el Anexo D.

Cada tablero tiene filtros de año y categoría, y el primero también de canal. Los hicimos con parámetros de Tableau [8] porque un parámetro filtra al mismo tiempo las dos fuentes de datos del libro, y los aplicamos como filtro de contexto para que el ranking de productos se calcule dentro de lo filtrado. Al pasar el cursor por cualquier barra o punto se ve el detalle del valor.

8.1 Tablero 1: rentabilidad de la venta

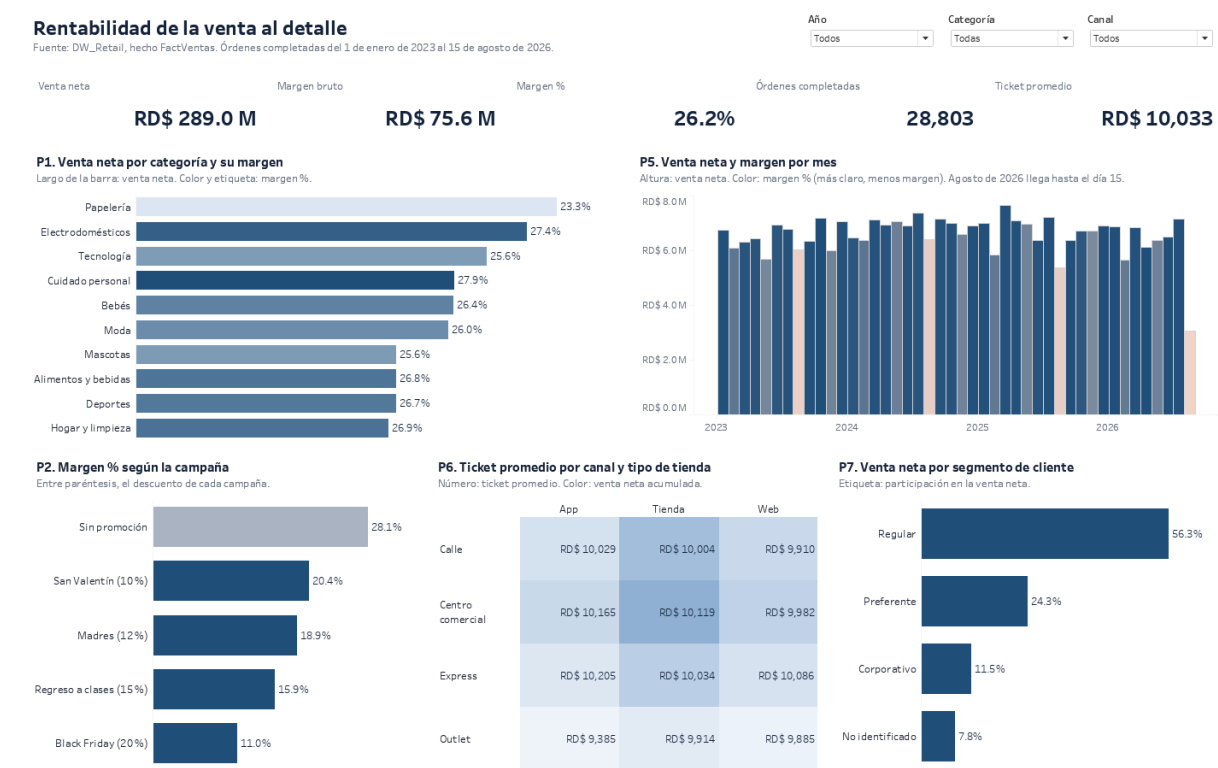


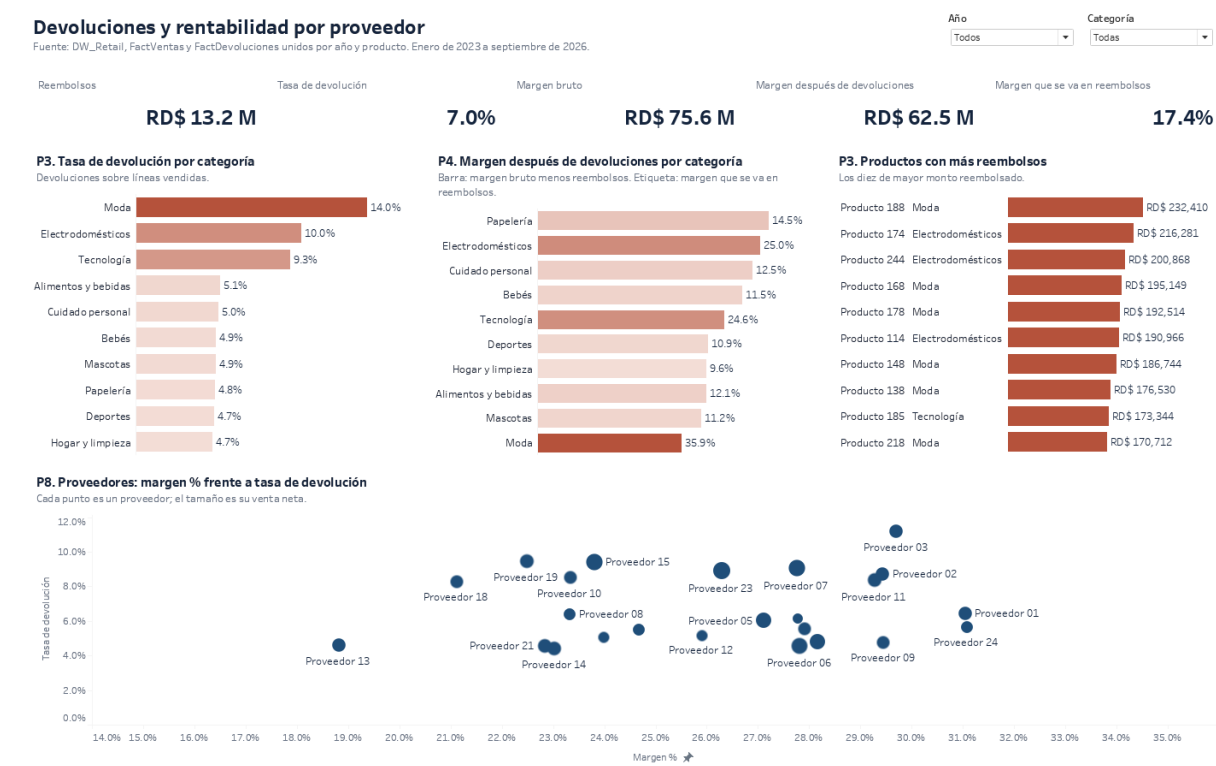
Figura 5. Tablero "Rentabilidad de la venta", sin filtros.

Las cifras de arriba resumen el período: venta neta de RD\$289.0 millones, margen bruto de RD\$75.6 millones (26.2%), 28,803 órdenes y un ticket promedio de RD\$10,033. Para P1, el gráfico de categorías muestra que Papelería es la que más vende (RD\$38.7 millones) pero la de menor margen (23.3%), mientras que Cuidado personal (27.9%) y Electrodomésticos (27.4%) son las más rentables.

Para P2, el margen cae a medida que la campaña da más descuento: San Valentín (10%) deja 20.4%, Madres (12%) 18.9%, Regreso a clases (15%) 15.9% y Black Friday (20%) apenas 11.0%, frente a 28.1% sin promoción. Como el ticket con promoción es menor, el descuento no se está compensando con más venta.

Para P5, la venta mensual se mueve entre RD\$5.4 y 7.7 millones, con un promedio de RD\$6.65 millones, y los meses más altos fueron marzo de 2025 y julio de 2024 y de 2025. Donde sí se ve un patrón es en el margen: agosto de cada año baja a cerca de 16% (15.8% en 2023 y 16.1% en 2024 y 2025), cuando corre la campaña de Regreso a clases, y noviembre de 2025, con Black Friday, baja a 22.7%. En el gráfico esos meses aparecen en un tono más claro. La última barra es baja porque agosto de 2026 solo tiene datos hasta el día 15. Para P6, el ticket promedio va de RD\$9,385 a RD\$10,205 y el margen de 25.8% a 26.5% en todas las combinaciones de canal y tipo de tienda; la mayor parte de la venta ocurre en tiendas físicas de centro comercial. Para P7, el segmento Regular aporta el 56.3% de la venta, Preferente el 24.3% y Corporativo el 11.5%, y el 7.8% de la venta no tiene cliente identificado.

8.2 Tablero 2: devoluciones y proveedores



En este tablero se ve que los reembolsos suman RD\$13.2 millones, con una tasa de devolución de 7.0%, y que después de restarlos el margen baja de RD\$75.6 a 62.5 millones: el 17.4% del margen se va en reembolsos. Para P3, Moda (14.0%), Electrodomésticos (10.0%) y Tecnología (9.3%) tienen tasas que duplican a las demás categorías, que están entre 4.7% y 5.1%, y los diez productos con más dinero reembolsado son de esas mismas categorías; el primero es el Producto 188, de Moda, con RD\$232,410.

Para P4, Moda pierde el 35.9% de su margen en reembolsos, Electrodomésticos el 25.0% y Tecnología el 24.6%, mientras que Papelería, a pesar de tener el margen más bajo, solo pierde el 14.5%. Para P8, el gráfico de dispersión muestra que el Proveedor 03 tiene uno de los márgenes más altos (29.7%) pero también la mayor tasa de devolución (11.2%), que el Proveedor 13 tiene el margen más bajo (18.8%) y que los proveedores 24 y 01 combinan márgenes por encima de 31% con devoluciones bajas.

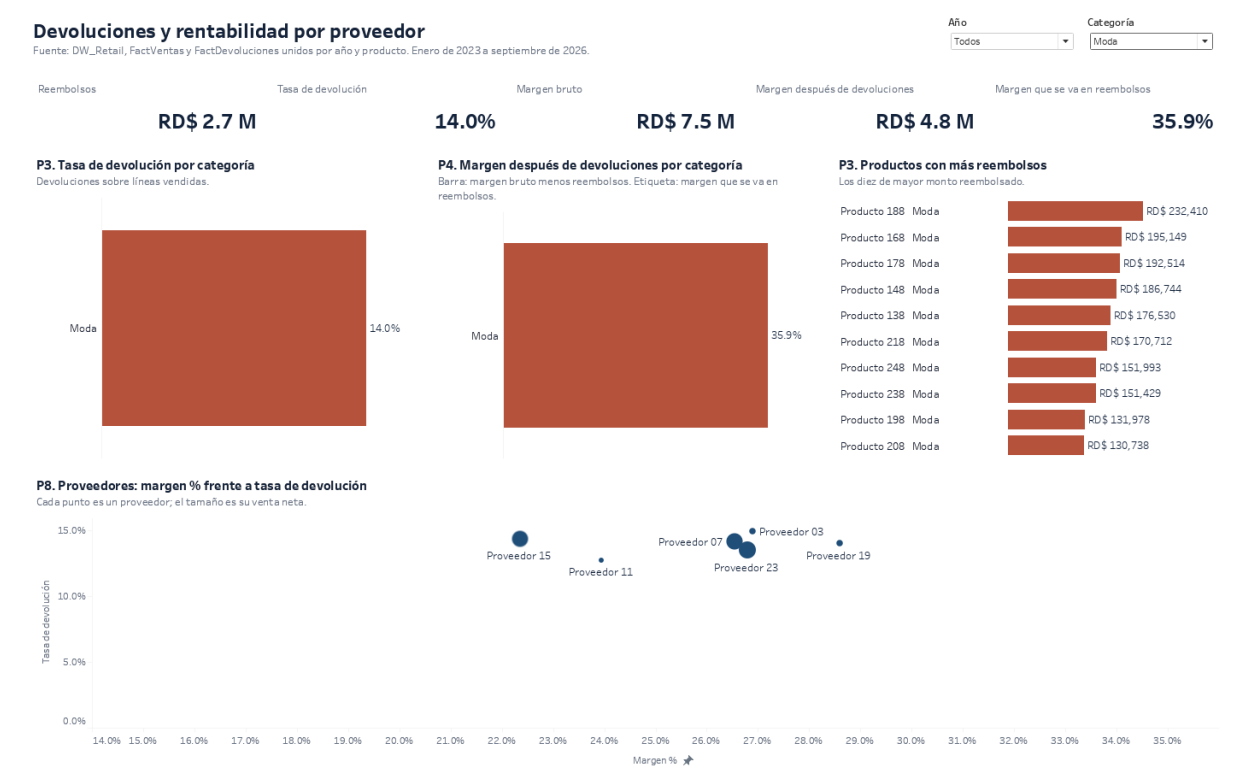


Figura 7. El mismo tablero con el filtro de categoría en Moda.

La Figura 7 muestra la interacción. Al elegir Moda en el filtro de categoría, todas las vistas se recalculan: la tasa de devolución de la categoría (14.0%), el margen que pierde (35.9%), los diez productos de Moda con más reembolsos y los proveedores que la abastecen.

8.3 Relación entre las preguntas y el dashboard

La Tabla 13 resume dónde se responde cada pregunta de análisis y con qué medidas y dimensiones del Data Warehouse, para mostrar que todas quedaron cubiertas.

Tabla 13. Preguntas de análisis y vista del dashboard que las responde.

Pregunta	Tablero y vista	Medidas	Dimensiones
P1	Tablero 1, venta neta por categoría y su margen	MontoNeto, margen %	Producto (categoría)
P2	Tablero 1, margen % según la campaña	MargenBruto, MontoNeto	Promoción
P3	Tablero 2, tasa de devolución por categoría y productos con más reembolsos	Devoluciones, MontoReembolso	Producto
P4	Tablero 2, margen después de devoluciones	MargenBruto - MontoReembolso	Producto (categoría)
P5	Tablero 1, venta neta y margen por mes	MontoNeto, margen %	Fecha (año, mes)
P6	Tablero 1, ticket promedio por canal y tipo de tienda	MontoNeto, órdenes	Canal, Tienda
P7	Tablero 1, venta neta por segmento de cliente	MontoNeto	Cliente (segmento)
P8	Tablero 2, proveedores: margen % frente a tasa de devolución	Margen %, tasa de devolución	Producto (proveedor)

Los filtros de año, categoría y canal se aplican a todas las vistas, así que cada pregunta también se puede responder para un año, una categoría o un canal en particular.

9. Justificación de las principales decisiones de diseño

Estas son las decisiones que más influyeron en la solución y por qué las tomamos:

- **Grano en la línea de orden completada.** Si el hecho estuviera a nivel de orden se perdería el producto, que es el eje de P1, P3 y P4.
- **Tres hechos separados.** El pago es por orden y la devolución es por línea; meterlos en la misma tabla que la venta duplicaría montos y rompería la aditividad.
- **Excluir las órdenes canceladas.** No son ventas efectivas; incluirlas sumaría RD\$12.1 millones que nunca se cobraron.
- **Miembros con clave -1.** Si el cliente o la promoción quedaran en nulo, 2,407 órdenes no se podrían cruzar con la dimensión Cliente y no se podría comparar con y sin promoción.
- **Esquema estrella y no copo de nieve.** Tener la categoría y el proveedor dentro de DimProducto reduce las uniones y hace el modelo más fácil de usar.

- **Medidas derivadas calculadas en el ETL.** Quedan listas en el Data Warehouse para cualquier herramienta, y el dashboard solo tiene que sumarlas.
- **Razones calculadas en el dashboard.** Un porcentaje guardado no se puede sumar ni promediar bien cuando se agrupa.
- **Filtros con parámetros.** Un parámetro filtra las dos fuentes del dashboard a la vez y permite que el ranking de productos se calcule dentro del filtro.

10. Conclusiones

La solución permite ver en un solo lugar lo que antes estaba separado entre ventas, descuentos y devoluciones. Con eso se responde el problema planteado: la empresa no puede juzgar sus categorías y promociones solo por lo que venden.

Los resultados más importantes para la toma de decisiones son tres. Las promociones profundas no se justifican con estos datos, porque Black Friday deja 11.0% de margen y no sube el ticket, así que conviene limitar los descuentos grandes a las categorías con más margen. Papelería necesita una revisión de precios o de costos con sus proveedores, porque es la categoría que más vende con el menor margen. Y las devoluciones de Moda, Electrodomésticos y Tecnología son la mayor fuente de pérdida de margen, por lo que los productos del ranking del segundo tablero son el punto de partida para revisar la calidad y la información de los productos, junto con proveedores como el 03. En cambio, el canal y el tipo de tienda casi no cambian la rentabilidad, y la caída del margen cada agosto confirma que el efecto de las promociones se ve incluso en la evolución mensual.

En cuanto al proceso, lo que más aprendimos fue la importancia de definir bien el grano desde el principio: casi todas las decisiones posteriores, desde separar los hechos hasta la forma de calcular el margen después de devoluciones, salieron de esa definición. También quedó claro que hay que verificar las cifras en cada etapa, porque problemas como el script que cambia en cada ejecución o las diferencias de formato entre computadoras solo aparecieron al comparar los resultados.

Referencias

- [1] R. Kimball y M. Ross, *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*, 3.a ed. Indianapolis, IN, EE. UU.: Wiley, 2013.

- [2] R. Sherman, *Business Intelligence Guidebook: From Data Integration to Analytics*. Waltham, MA, EE. UU.: Morgan Kaufmann, 2014.
- [3] Microsoft, "BULK INSERT (Transact-SQL)," Microsoft Learn. [En línea]. Disponible: <https://learn.microsoft.com/en-us/sql/t-sql/statements/bulk-insert-transact-sql>
- [4] Microsoft, "SET DATEFIRST (Transact-SQL)," Microsoft Learn. [En línea]. Disponible: <https://learn.microsoft.com/en-us/sql/t-sql/statements/set-datefirst-transact-sql>
- [5] Tableau, "Extract Your Data," Tableau Desktop Help. [En línea]. Disponible: https://help.tableau.com/current/pro/desktop/en-us/extracting_data.htm
- [6] Tableau, "Aggregate, Join, or Union Data," Tableau Prep Help. [En línea]. Disponible: https://help.tableau.com/current/prep/en-us/prep_combine.htm
- [7] Tableau, "Save and Share Tableau Prep Work," Tableau Prep Help. [En línea]. Disponible: https://help.tableau.com/current/prep/en-us/prep_save_share.htm
- [8] Tableau, "Create Parameters," Tableau Desktop Help. [En línea]. Disponible: https://help.tableau.com/current/pro/desktop/en-us/parameters_create.htm

Anexo A. Script de creación del Data Warehouse

Contenido completo de DW_Retail.sql. Crea la base DW_Retail con las ocho dimensiones, los tres hechos, sus claves primarias y foráneas, restricciones e índices, llena DimFecha e inserta los miembros con clave -1.

```
/*
=====
ICC-321 Inteligencia de Negocios, Práctica 1
Actividad 4: Implementación del Data Warehouse

Estudiantes: Elías De La Cruz Jiménez (ID 10155971), Enger Douglas (ID 10144675)
Motor: Microsoft SQL Server
Base fuente: BI_Practica_Retail

Modelo dimensional de Kimball:
- FactVentas          grano: línea de detalle de una orden completada
- FactDevoluciones    grano: devolución de una línea de detalle
- FactPagos           grano: pago de una orden
Las claves subrogadas se asignan a partir del identificador del origen y se
reserva la clave -1 para los miembros desconocidos («No identificado»,
«Sin promoción»).
=====
*/

USE master;
GO

IF DB_ID(N'DW_Retail') IS NOT NULL
BEGIN
    ALTER DATABASE DW_Retail SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
    DROP DATABASE DW_Retail;
END;
GO

CREATE DATABASE DW_Retail;
GO

USE DW_Retail;
GO

/* =====
1. DIMENSIONES
===== */

CREATE TABLE dbo.DimFecha (
    FechaKey          int          NOT NULL PRIMARY KEY,    -- formato AAAAMMDD
    Fecha             date         NOT NULL,
    Anio              smallint     NOT NULL,
    Trimestre         tinyint      NOT NULL,
    NombreTrimestre   varchar(10)  NOT NULL,
    Mes               tinyint      NOT NULL,
    NombreMes         varchar(15)  NOT NULL,
    AnioMes           int          NOT NULL,              -- formato AAAAMM
    DiaMes            tinyint      NOT NULL,
    DiaSemana         tinyint      NOT NULL,
    NombreDiaSemana   varchar(12)  NOT NULL,
    SemanaAnio        tinyint      NOT NULL,
    EsFinDeSemana     bit          NOT NULL
);

CREATE TABLE dbo.DimProducto (
    ProductoKey       int          NOT NULL PRIMARY KEY,
    ProductoID        int          NOT NULL,              -- clave natural del origen
    SKU               varchar(20)  NOT NULL,
    NombreProducto    varchar(120) NOT NULL,
    Marca             varchar(60)  NOT NULL,
    Categoria         varchar(80)  NOT NULL,
    Proveedor         varchar(120) NOT NULL,
    PaisProveedor     varchar(60)  NOT NULL,
    CostoUnitario     decimal(12,2) NOT NULL,
    PrecioLista       decimal(12,2) NOT NULL,
    RangoPrecio       varchar(20)  NOT NULL,
    FechaLanzamiento  date         NOT NULL,
    Activo            bit          NOT NULL
);

CREATE TABLE dbo.DimCliente (
    ClienteKey        int          NOT NULL PRIMARY KEY,    -- -1 = No identificado
    ClienteID         int          NULL,
    NombreCompleto    varchar(130) NOT NULL,
```

```

Sexo          varchar(20) NOT NULL,          -- M, F o «No especificado»
GrupoEdad     varchar(20) NOT NULL,
Ciudad        varchar(80) NOT NULL,
Provincia     varchar(80) NOT NULL,
Segmento      varchar(30) NOT NULL,
AnioRegistro  smallint  NULL
);

CREATE TABLE dbo.DimTienda (
    TiendaKey   int          NOT NULL PRIMARY KEY,
    TiendaID    int          NOT NULL,
    NombreTienda varchar(100) NOT NULL,
    Ciudad      varchar(80)  NOT NULL,
    Provincia   varchar(80)  NOT NULL,
    TipoTienda  varchar(30)  NOT NULL,
    AnioApertura smallint    NOT NULL
);

CREATE TABLE dbo.DimPromocion (
    PromocionKey int          NOT NULL PRIMARY KEY, -- -1 = Sin promoción
    PromocionID  int          NULL,
    NombrePromocion varchar(100) NOT NULL,
    PorcentajeDto decimal(5,2) NOT NULL,
    CanalPromocion varchar(20) NOT NULL,
    FechaInicio  date         NULL,
    FechaFin     date         NULL
);

CREATE TABLE dbo.DimCanal (
    CanalKey    int          NOT NULL PRIMARY KEY,
    CanalVenta  varchar(20)  NOT NULL
);

CREATE TABLE dbo.DimMotivoDevolucion (
    MotivoKey   int          NOT NULL PRIMARY KEY,
    Motivo      varchar(50)  NOT NULL
);

CREATE TABLE dbo.DimMetodoPago (
    MetodoPagoKey int          NOT NULL PRIMARY KEY,
    MetodoPago    varchar(30)  NOT NULL
);
GO

/* =====
2. TABLAS DE HECHOS
===== */

/* Grano: una línea de detalle de una orden de venta COMPLETADA */
CREATE TABLE dbo.FactVentas (
    VentaKey      bigint          NOT NULL PRIMARY KEY, -- = DetalleOrdenID del origen
    FechaKey      int             NOT NULL,
    ProductoKey   int             NOT NULL,
    ClienteKey    int             NOT NULL,
    TiendaKey     int             NOT NULL,
    PromocionKey  int             NOT NULL,
    CanalKey      int             NOT NULL,
    OrdenID      bigint          NOT NULL,
    CantidadVendida int          NOT NULL,
    PrecioUnitario decimal(12,2) NOT NULL,
    MontoBruto    decimal(14,2) NOT NULL,
    DescuentoMonto decimal(14,2) NOT NULL,
    MontoNeto     decimal(14,2) NOT NULL,
    CostoTotal    decimal(14,2) NOT NULL,
    MargenBruto   decimal(14,2) NOT NULL,
    CONSTRAINT FK_FactVentas_Fecha FOREIGN KEY (FechaKey) REFERENCES dbo.DimFecha(FechaKey),
    CONSTRAINT FK_FactVentas_Producto FOREIGN KEY (ProductoKey) REFERENCES dbo.DimProducto(ProductoKey),
    CONSTRAINT FK_FactVentas_Cliente FOREIGN KEY (ClienteKey) REFERENCES dbo.DimCliente(ClienteKey),
    CONSTRAINT FK_FactVentas_Tienda FOREIGN KEY (TiendaKey) REFERENCES dbo.DimTienda(TiendaKey),
    CONSTRAINT FK_FactVentas_Promocion FOREIGN KEY (PromocionKey) REFERENCES dbo.DimPromocion(PromocionKey),
    CONSTRAINT FK_FactVentas_Canal FOREIGN KEY (CanalKey) REFERENCES dbo.DimCanal(CanalKey),
    CONSTRAINT CK_FactVentas_Cantidad CHECK (CantidadVendida > 0)
);

/* Grano: una devolución de una línea de detalle */
CREATE TABLE dbo.FactDevoluciones (
    DevolucionKey bigint          NOT NULL PRIMARY KEY, -- = DevolucionID del origen
    FechaKey      int             NOT NULL,
    ProductoKey   int             NOT NULL,
    ClienteKey    int             NOT NULL,
    TiendaKey     int             NOT NULL,
    MotivoKey     int             NOT NULL,
    DetalleOrdenID bigint          NOT NULL,
    CantidadDevoluelta int          NOT NULL,
    MontoReembolso decimal(14,2) NOT NULL,
    CONSTRAINT FK_FactDev_Fecha FOREIGN KEY (FechaKey) REFERENCES dbo.DimFecha(FechaKey),
    CONSTRAINT FK_FactDev_Producto FOREIGN KEY (ProductoKey) REFERENCES dbo.DimProducto(ProductoKey),
    CONSTRAINT FK_FactDev_Cliente FOREIGN KEY (ClienteKey) REFERENCES dbo.DimCliente(ClienteKey),

```

```

        CONSTRAINT FK_FactDev_Tienda FOREIGN KEY (TiendaKey) REFERENCES dbo.DimTienda(TiendaKey),
        CONSTRAINT FK_FactDev_Motivo FOREIGN KEY (MotivoKey) REFERENCES dbo.DimMotivoDevolucion(MotivoKey),
        CONSTRAINT CK_FactDev_Cantidad CHECK (CantidadDevoluelta > 0)
    );

/* Grano: el pago de una orden */
CREATE TABLE dbo.FactPagos (
    PagoKey bigint NOT NULL PRIMARY KEY, -- = PagoID del origen
    FechaKey int NOT NULL,
    ClienteKey int NOT NULL,
    TiendaKey int NOT NULL,
    CanalKey int NOT NULL,
    MetodoPagoKey int NOT NULL,
    OrdenID bigint NOT NULL, -- dimensión degenerada
    MontoPagado decimal(14,2) NOT NULL,
    CONSTRAINT FK_FactPagos_Fecha FOREIGN KEY (FechaKey) REFERENCES dbo.DimFecha(FechaKey),
    CONSTRAINT FK_FactPagos_Cliente FOREIGN KEY (ClienteKey) REFERENCES dbo.DimCliente(ClienteKey),
    CONSTRAINT FK_FactPagos_Tienda FOREIGN KEY (TiendaKey) REFERENCES dbo.DimTienda(TiendaKey),
    CONSTRAINT FK_FactPagos_Canal FOREIGN KEY (CanalKey) REFERENCES dbo.DimCanal(CanalKey),
    CONSTRAINT FK_FactPagos_Metodo FOREIGN KEY (MetodoPagoKey) REFERENCES dbo.DimMetodoPago(MetodoPagoKey)
);
GO

/* Índices sobre las claves foráneas de los hechos */
CREATE INDEX IX_FactVentas_Fecha ON dbo.FactVentas(FechaKey);
CREATE INDEX IX_FactVentas_Producto ON dbo.FactVentas(ProductoKey);
CREATE INDEX IX_FactVentas_Cliente ON dbo.FactVentas(ClienteKey);
CREATE INDEX IX_FactVentas_Tienda ON dbo.FactVentas(TiendaKey);
CREATE INDEX IX_FactVentas_Promocion ON dbo.FactVentas(PromocionKey);
CREATE INDEX IX_FactDev_Fecha ON dbo.FactDevoluciones(FechaKey);
CREATE INDEX IX_FactDev_Producto ON dbo.FactDevoluciones(ProductoKey);
CREATE INDEX IX_FactPagos_Fecha ON dbo.FactPagos(FechaKey);
GO

/* =====
3. CARGA DE LA DIMENSIÓN DE TIEMPO
Se genera de forma independiente, con todos los días del período 2023-2026.
===== */

/* El día de la semana y la semana del año dependen de DATEFIRST; se fija el
domingo como primer día para que el resultado no cambie según el servidor. */
SET DATEFIRST 7;

WITH Numeros AS (
    SELECT TOP (DATEDIFF(day, '2023-01-01', '2026-12-31') + 1)
        ROW_NUMBER() OVER (ORDER BY (SELECT NULL)) - 1 AS n
    FROM sys.all_objects a CROSS JOIN sys.all_objects b
),
Fechas AS (
    SELECT DATEADD(day, n, CAST('2023-01-01' AS date)) AS Fecha FROM Numeros
)
INSERT INTO dbo.DimFecha
(FechaKey, Fecha, Anio, Trimestre, NombreTrimestre, Mes, NombreMes, AnioMes,
DiaMes, DiaSemana, NombreDiaSemana, SemanaAnio, EsFinDeSemana)
SELECT
    CONVERT(int, FORMAT(Fecha, 'yyyyMMdd')),
    Fecha,
    YEAR(Fecha),
    DATEPART(quarter, Fecha),
    CONCAT('T', DATEPART(quarter, Fecha)),
    MONTH(Fecha),
    CHOOSE(MONTH(Fecha), 'Enero', 'Febrero', 'Marzo', 'Abril', 'Mayo', 'Junio',
        'Julio', 'Agosto', 'Septiembre', 'Octubre', 'Noviembre', 'Diciembre'),
    CONVERT(int, FORMAT(Fecha, 'yyyyMM')),
    DAY(Fecha),
    DATEPART(weekday, Fecha),
    CHOOSE(DATEPART(weekday, Fecha), 'Domingo', 'Lunes', 'Martes', 'Miercoles', 'Jueves', 'Viernes', 'Sabado'),
    DATEPART(week, Fecha),
    CASE WHEN DATEPART(weekday, Fecha) IN (1, 7) THEN 1 ELSE 0 END
FROM Fechas;
GO

/* =====
4. MIEMBROS ESPECIALES PARA LOS NULOS DEL ORIGEN
===== */

INSERT INTO dbo.DimCliente
(ClienteKey, ClienteID, NombreCompleto, Sexo, GrupoEdad, Ciudad, Provincia, Segmento, AnioRegistro)
VALUES
(-1, NULL, 'No identificado', 'No especificado', 'No aplica', 'No aplica', 'No aplica', 'No identificado', NULL);

INSERT INTO dbo.DimPromocion
(PromocionKey, PromocionID, NombrePromocion, PorcentajeDto, CanalPromocion, FechaInicio, FechaFin)
VALUES
(-1, NULL, 'Sin promocion', 0.00, 'No aplica', NULL, NULL);
GO

/* =====

```

5. VERIFICACIÓN

```
===== */
SELECT 'DimFecha' AS Tabla, COUNT(*) AS Filas, MIN(Fecha) AS Desde, MAX(Fecha) AS Hasta FROM dbo.DimFecha
UNION ALL
SELECT 'DimCliente (miembro -1)', COUNT(*), NULL, NULL FROM dbo.DimCliente WHERE ClienteKey = -1
UNION ALL
SELECT 'DimPromocion (miembro -1)', COUNT(*), NULL, NULL FROM dbo.DimPromocion WHERE PromocionKey = -1;
GO
```

Anexo B. Script de carga del Data Warehouse

Contenido completo de Cargar_DW.sql. Se ejecuta en modo SQLCMD después de correr el flujo de Tableau Prep y carga los diez archivos CSV en las tablas del Data Warehouse.

```
/*
=====
ICC-321 Inteligencia de Negocios, Práctica 1
Carga del Data Warehouse con la salida del flujo de Tableau Prep

Estudiantes: Elías De La Cruz Jiménez (ID 10155971), Enger Douglas (ID 10144675)

Requisito previo: haber ejecutado DW_Retail.sql (crea la base y DimFecha)
y el flujo ETL_DW_Retail.tfl (genera los CSV en 5_ETL/CSV_salida).

Se ejecuta en modo SQLCMD (sqlcmd, o SSMS con Consulta > Modo SQLCMD).
La variable RutaCSV indica la carpeta de los CSV vista desde el servidor,
terminada en separador; se cambia en la línea :setvar de abajo.

Los CSV se cargan en tablas temporales de staging, porque Tableau Prep
escribe las columnas en orden alfabético, y de ahí se insertan en las tablas
definitivas con el tipo de dato correcto. Tableau Prep en Windows termina las
líneas con CRLF y en macOS con LF; después de cada carga se quita el retorno
de carro que queda en la última columna, así el script sirve con ambos.
=====
*/

USE DW_Retail;
GO

:setvar RutaCSV "/datos/"

SET NOCOUNT ON;
/* Tableau Prep escribe las fechas como mes/día/año; así se leen igual
sin importar el idioma configurado en el servidor. */
SET DATEFORMAT mdy;

/* Las tablas de hechos se vacían primero por las claves foráneas */
DELETE FROM dbo.FactVentas;
DELETE FROM dbo.FactDevoluciones;
DELETE FROM dbo.FactPagos;
DELETE FROM dbo.DimProducto;
DELETE FROM dbo.DimTienda;
DELETE FROM dbo.DimCanal;
DELETE FROM dbo.DimMotivoDevolucion;
DELETE FROM dbo.DimMetodoPago;
DELETE FROM dbo.DimCliente WHERE ClienteKey <> -1; -- se conserva «No identificado»
DELETE FROM dbo.DimPromocion WHERE PromocionKey <> -1; -- se conserva «Sin promocion»
GO

/* =====
DIMENSIONES
===== */

CREATE TABLE #DimProducto (Activo varchar(10), Categoria varchar(80), CostoUnitario varchar(30),
FechaLanzamiento varchar(30), Marca varchar(60), NombreProducto varchar(120), PaisProveedor varchar(60),
PrecioLista varchar(30), ProductoID varchar(20), ProductoKey varchar(20), Proveedor varchar(120),
RangoPrecio varchar(20), SKU varchar(20));
BULK INSERT #DimProducto FROM '$(RutaCSV)DimProducto.csv'
WITH (FORMAT='CSV', FIRSTROW=2, FIELDTERMINATOR=',', ROWTERMINATOR='0x0a');
UPDATE #DimProducto SET SKU = REPLACE(SKU, CHAR(13), '');
INSERT INTO dbo.DimProducto (ProductoKey, ProductoID, SKU, NombreProducto, Marca, Categoria, Proveedor,
PaisProveedor, CostoUnitario, PrecioLista, RangoPrecio, FechaLanzamiento, Activo)
SELECT CAST(ProductoKey AS int), CAST(ProductoID AS int), SKU, NombreProducto, Marca, Categoria, Proveedor,
PaisProveedor, CAST(CostoUnitario AS decimal(12,2)), CAST(PrecioLista AS decimal(12,2)), RangoPrecio,
CAST(FechaLanzamiento AS date), CAST(CAST(Activo AS int) AS bit)
FROM #DimProducto;

CREATE TABLE #DimCliente (AnioRegistro varchar(10), Ciudad varchar(80), ClienteID varchar(20), ClienteKey varchar(20),
GrupoEdad varchar(20), NombreCompleto varchar(130), Provincia varchar(80), Segmento varchar(30), Sexo
varchar(20));
BULK INSERT #DimCliente FROM '$(RutaCSV)DimCliente.csv'
WITH (FORMAT='CSV', FIRSTROW=2, FIELDTERMINATOR=',', ROWTERMINATOR='0x0a');
UPDATE #DimCliente SET Sexo = REPLACE(Sexo, CHAR(13), '');
INSERT INTO dbo.DimCliente (ClienteKey, ClienteID, NombreCompleto, Sexo, GrupoEdad, Ciudad, Provincia, Segmento,
AnioRegistro)
SELECT CAST(ClienteKey AS int), CAST(ClienteID AS int), NombreCompleto, Sexo, GrupoEdad, Ciudad, Provincia,
Segmento, CAST(NULLIF(AnioRegistro, '') AS smallint)
FROM #DimCliente;

CREATE TABLE #DimTienda (AnioApertura varchar(10), Ciudad varchar(80), NombreTienda varchar(100),
Provincia varchar(80), TiendaID varchar(20), TiendaKey varchar(20), TipoTienda varchar(30));
BULK INSERT #DimTienda FROM '$(RutaCSV)DimTienda.csv'
```

```

        WITH (FORMAT='CSV', FIRSTROW=2, FIELDTERMINATOR=',', ROWTERMINATOR='0x0a');
UPDATE #DimTienda SET TipoTienda = REPLACE(TipoTienda, CHAR(13), '');
INSERT INTO dbo.DimTienda (TiendaKey, TiendaID, NombreTienda, Ciudad, Provincia, TipoTienda, AnioApertura)
SELECT CAST(TiendaKey AS int), CAST(TiendaID AS int), NombreTienda, Ciudad, Provincia, TipoTienda,
        CAST(AnioApertura AS smallint)
FROM #DimTienda;

CREATE TABLE #DimPromocion (CanalPromocion varchar(20), FechaFin varchar(30), FechaInicio varchar(30),
        NombrePromocion varchar(100), PorcentajeDto varchar(20), PromocionID varchar(20), PromocionKey varchar(20));
BULK INSERT #DimPromocion FROM '$(RutaCSV)DimPromocion.csv'
        WITH (FORMAT='CSV', FIRSTROW=2, FIELDTERMINATOR=',', ROWTERMINATOR='0x0a');
UPDATE #DimPromocion SET PromocionKey = REPLACE(PromocionKey, CHAR(13), '');
INSERT INTO dbo.DimPromocion (PromocionKey, PromocionID, NombrePromocion, PorcentajeDto, CanalPromocion, FechaInicio,
        FechaFin)
SELECT CAST(PromocionKey AS int), CAST(PromocionID AS int), NombrePromocion, CAST(PorcentajeDto AS decimal(5,2)),
        CanalPromocion, CAST(FechaInicio AS date), CAST(FechaFin AS date)
FROM #DimPromocion;

CREATE TABLE #DimCanal (CanalKey varchar(20), CanalVenta varchar(20));
BULK INSERT #DimCanal FROM '$(RutaCSV)DimCanal.csv'
        WITH (FORMAT='CSV', FIRSTROW=2, FIELDTERMINATOR=',', ROWTERMINATOR='0x0a');
UPDATE #DimCanal SET CanalVenta = REPLACE(CanalVenta, CHAR(13), '');
INSERT INTO dbo.DimCanal (CanalKey, CanalVenta) SELECT CAST(CanalKey AS int), CanalVenta FROM #DimCanal;

CREATE TABLE #DimMotivo (Motivo varchar(50), MotivoKey varchar(20));
BULK INSERT #DimMotivo FROM '$(RutaCSV)DimMotivoDevolucion.csv'
        WITH (FORMAT='CSV', FIRSTROW=2, FIELDTERMINATOR=',', ROWTERMINATOR='0x0a');
UPDATE #DimMotivo SET MotivoKey = REPLACE(MotivoKey, CHAR(13), '');
INSERT INTO dbo.DimMotivoDevolucion (MotivoKey, Motivo) SELECT CAST(MotivoKey AS int), Motivo FROM #DimMotivo;

CREATE TABLE #DimMetodoPago (MetodoPago varchar(30), MetodoPagoKey varchar(20));
BULK INSERT #DimMetodoPago FROM '$(RutaCSV)DimMetodoPago.csv'
        WITH (FORMAT='CSV', FIRSTROW=2, FIELDTERMINATOR=',', ROWTERMINATOR='0x0a');
UPDATE #DimMetodoPago SET MetodoPagoKey = REPLACE(MetodoPagoKey, CHAR(13), '');
INSERT INTO dbo.DimMetodoPago (MetodoPagoKey, MetodoPago) SELECT CAST(MetodoPagoKey AS int), MetodoPago FROM
        #DimMetodoPago;
GO

/* =====
HECHOS
===== */

CREATE TABLE #FactVentas (CanalKey varchar(20), CantidadVendida varchar(20), ClienteKey varchar(20),
        CostoTotal varchar(30), DescuentoMonto varchar(30), FechaKey varchar(20), MargenBruto varchar(30),
        MontoBruto varchar(30), MontoNeto varchar(30), OrdenID varchar(20), PrecioUnitario varchar(30),
        ProductoKey varchar(20), PromocionKey varchar(20), TiendaKey varchar(20), VentaKey varchar(20));
BULK INSERT #FactVentas FROM '$(RutaCSV)FactVentas.csv'
        WITH (FORMAT='CSV', FIRSTROW=2, FIELDTERMINATOR=',', ROWTERMINATOR='0x0a', BATCHSIZE=20000);
UPDATE #FactVentas SET VentaKey = REPLACE(VentaKey, CHAR(13), '');
INSERT INTO dbo.FactVentas (VentaKey, FechaKey, ProductoKey, ClienteKey, TiendaKey, PromocionKey, CanalKey,
        OrdenID, CantidadVendida, PrecioUnitario, MontoBruto, DescuentoMonto, MontoNeto,
        CostoTotal, MargenBruto)
SELECT CAST(VentaKey AS bigint), CAST(FechaKey AS int), CAST(ProductoKey AS int), CAST(ClienteKey AS int),
        CAST(TiendaKey AS int), CAST(PromocionKey AS int), CAST(CanalKey AS int), CAST(OrdenID AS bigint),
        CAST(CantidadVendida AS int), CAST(PrecioUnitario AS decimal(12,2)), CAST(MontoBruto AS decimal(14,2)),
        CAST(DescuentoMonto AS decimal(14,2)), CAST(MontoNeto AS decimal(14,2)), CAST(CostoTotal AS decimal(14,2)),
        CAST(MargenBruto AS decimal(14,2))
FROM #FactVentas;

CREATE TABLE #FactDevoluciones (CantidadDevolucion varchar(20), ClienteKey varchar(20), DetalleOrdenID varchar(20),
        DevolucionKey varchar(20), FechaKey varchar(20), MontoReembolso varchar(30), MotivoKey varchar(20),
        ProductoKey varchar(20), TiendaKey varchar(20));
BULK INSERT #FactDevoluciones FROM '$(RutaCSV)FactDevoluciones.csv'
        WITH (FORMAT='CSV', FIRSTROW=2, FIELDTERMINATOR=',', ROWTERMINATOR='0x0a');
UPDATE #FactDevoluciones SET TiendaKey = REPLACE(TiendaKey, CHAR(13), '');
INSERT INTO dbo.FactDevoluciones (DevolucionKey, FechaKey, ProductoKey, ClienteKey, TiendaKey, MotivoKey,
        DetalleOrdenID, CantidadDevolucion, MontoReembolso)
SELECT CAST(DevolucionKey AS bigint), CAST(FechaKey AS int), CAST(ProductoKey AS int), CAST(ClienteKey AS int),
        CAST(TiendaKey AS int), CAST(MotivoKey AS int), CAST(DetalleOrdenID AS bigint),
        CAST(CantidadDevolucion AS int), CAST(MontoReembolso AS decimal(14,2))
FROM #FactDevoluciones;

CREATE TABLE #FactPagos (CanalKey varchar(20), ClienteKey varchar(20), FechaKey varchar(20),
        MetodoPagoKey varchar(20), MontoPagado varchar(30), OrdenID varchar(20), PagoKey varchar(20), TiendaKey
        varchar(20));
BULK INSERT #FactPagos FROM '$(RutaCSV)FactPagos.csv'
        WITH (FORMAT='CSV', FIRSTROW=2, FIELDTERMINATOR=',', ROWTERMINATOR='0x0a', BATCHSIZE=20000);
UPDATE #FactPagos SET TiendaKey = REPLACE(TiendaKey, CHAR(13), '');
INSERT INTO dbo.FactPagos (PagoKey, FechaKey, ClienteKey, TiendaKey, CanalKey, MetodoPagoKey, OrdenID, MontoPagado)
SELECT CAST(PagoKey AS bigint), CAST(FechaKey AS int), CAST(ClienteKey AS int), CAST(TiendaKey AS int),
        CAST(CanalKey AS int), CAST(MetodoPagoKey AS int), CAST(OrdenID AS bigint), CAST(MontoPagado AS decimal(14,2))
FROM #FactPagos;
GO

/* =====
VERIFICACIÓN DE LA CARGA
===== */

```

```

SELECT 'DimFecha' AS Tabla, COUNT(*) AS Filas FROM dbo.DimFecha
UNION ALL SELECT 'DimProducto', COUNT(*) FROM dbo.DimProducto
UNION ALL SELECT 'DimCliente', COUNT(*) FROM dbo.DimCliente
UNION ALL SELECT 'DimTienda', COUNT(*) FROM dbo.DimTienda
UNION ALL SELECT 'DimPromocion', COUNT(*) FROM dbo.DimPromocion
UNION ALL SELECT 'DimCanal', COUNT(*) FROM dbo.DimCanal
UNION ALL SELECT 'DimMotivoDevolucion', COUNT(*) FROM dbo.DimMotivoDevolucion
UNION ALL SELECT 'DimMetodoPago', COUNT(*) FROM dbo.DimMetodoPago
UNION ALL SELECT 'FactVentas', COUNT(*) FROM dbo.FactVentas
UNION ALL SELECT 'FactDevoluciones', COUNT(*) FROM dbo.FactDevoluciones
UNION ALL SELECT 'FactPagos', COUNT(*) FROM dbo.FactPagos;

SELECT CAST(SUM(MontoNeto) AS decimal(18,2)) AS VentaNeta,
       CAST(SUM(MargenBruto) AS decimal(18,2)) AS MargenBruto,
       SUM(CantidadVendida) AS Unidades,
       CAST(100.0 * SUM(MargenBruto) / SUM(MontoNeto) AS decimal(5,2)) AS MargenPct
FROM dbo.FactVentas;
GO

```

Anexo C. Detalle del flujo ETL en Tableau Prep

Todos los pasos del flujo ETL_DW_Retail en el orden en que aparecen en Tableau Prep, con cada operación y cada cálculo tal como están definidos en el flujo.

Tabla 14. Pasos y operaciones del flujo ETL_DW_Retail.

Paso	Tipo	Operación
Categoría	Entrada	Archivo Categoría.csv. Campos: CategoríaID, NombreCategoría
Proveedor	Entrada	Archivo Proveedor.csv. Campos: ProveedorID, NombreProveedor, País, FechaAlta
Producto	Entrada	Archivo Producto.csv. Campos: ProductoID, SKU, NombreProducto, CategoríaID, ProveedorID, Marca, CostoUnitario, PrecioLista, FechaLanzamiento, Activo
Tienda	Entrada	Archivo Tienda.csv. Campos: TiendaID, NombreTienda, Ciudad, Provincia, TipoTienda, FechaApertura
Cliente	Entrada	Archivo Cliente.csv. Campos: ClienteID, Nombre, Apellido, Sexo, FechaNacimiento, Ciudad, Provincia, Segmento, FechaRegistro
Promoción	Entrada	Archivo Promoción.csv. Campos: PromociónID, NombrePromoción, FechaInicio, FechaFin, PorcentajeDto, CanalPromoción
OrdenVenta	Entrada	Archivo OrdenVenta.csv. Campos: OrdenID, ClienteID, TiendaID, FechaOrden, CanalVenta, EstadoOrden, PromociónID
DetalleOrden	Entrada	Archivo DetalleOrden.csv. Campos: DetalleOrdenID, OrdenID, ProductoID, Cantidad, PrecioUnitario, DescuentoMonto
Pago	Entrada	Archivo Pago.csv. Campos: PagoID, OrdenID, FechaPago, MetodoPago, Monto
Devolución	Entrada	Archivo Devolución.csv. Campos: DevoluciónID, DetalleOrdenID, FechaDevolución, CantidadDevolución, Motivo, MontoReembolso
Producto + Categoría	Unión	Unión interna por [CategoríaID] = [CategoríaID]
Producto + Proveedor	Unión	Unión interna por [ProveedorID] = [ProveedorID]
DimProducto	Limpieza	Renombra NombreCategoría a Categoría
DimProducto	Limpieza	Renombra NombreProveedor a Proveedor
DimProducto	Limpieza	Renombra País a PaísProveedor
DimProducto	Limpieza	Campo ProductoKey = [ProductoID]
DimProducto	Limpieza	Campo RangoPrecio = IF [PrecioLista] < 800 THEN "Economico" ELSEIF [PrecioLista] < 1500 THEN "Medio" ELSE "Premium" END
DimProducto	Limpieza	Quita las columnas CategoríaID, ProveedorID, FechaAlta
Salida DimProducto	Salida	Escribe DimProducto.csv
DimCliente	Limpieza	Campo ClienteKey = [ClienteID]
DimCliente	Limpieza	Campo NombreCompleto = [Nombre] + " " + [Apellido]
DimCliente	Limpieza	Campo Sexo = IFNULL([Sexo], "No especificado")

Paso	Tipo	Operación
DimCliente	Limpieza	Campo Edad = DATEDIFF("year", [FechaNacimiento], #2026-08-15#) - IIF(DATEADD("year", DATEDIFF("year", [FechaNacimiento], #2026-08-15#), [FechaNacimiento]) > #2026-08-15#, 1, 0)
DimCliente	Limpieza	Campo GrupoEdad = IF [Edad] < 30 THEN "Menor de 30" ELSEIF [Edad] < 45 THEN "30 a 44" ELSEIF [Edad] < 60 THEN "45 a 59" ELSE "60 o mas" END
DimCliente	Limpieza	Campo AnioRegistro = YEAR([FechaRegistro])
DimCliente	Limpieza	Quita las columnas Nombre, Apellido, FechaNacimiento, FechaRegistro, Edad
Salida DimCliente	Salida	Escribe DimCliente.csv
DimTienda	Limpieza	Campo TiendaKey = [TiendaID]
DimTienda	Limpieza	Campo AnioApertura = YEAR([FechaApertura])
DimTienda	Limpieza	Quita las columnas FechaApertura
Salida DimTienda	Salida	Escribe DimTienda.csv
DimPromocion	Limpieza	Campo PromocionKey = [PromocionID]
Salida DimPromocion	Salida	Escribe DimPromocion.csv
Canales distintos	Agregación	Agrupar por CanalVenta para obtener los valores distintos
DimCanal	Limpieza	Campo CanalKey = CASE [CanalVenta] WHEN "Tienda" THEN 1 WHEN "Web" THEN 2 WHEN "App" THEN 3 ELSE 0 END
DimCanal	Limpieza	Quita las columnas Ordenes
Salida DimCanal	Salida	Escribe DimCanal.csv
Motivos distintos	Agregación	Agrupar por Motivo para obtener los valores distintos
DimMotivoDevolucion	Limpieza	Campo MotivoKey = CASE [Motivo] WHEN "Producto defectuoso" THEN 1 WHEN "Producto equivocado" THEN 2 WHEN "Cambio de opinion" THEN 3 WHEN "Danado en transporte" THEN 4 WHEN "Talla o presentacion" THEN 5 ELSE 0 END
DimMotivoDevolucion	Limpieza	Quita las columnas Devoluciones
Salida DimMotivoDevolucion	Salida	Escribe DimMotivoDevolucion.csv
Metodos de pago distintos	Agregación	Agrupar por MetodoPago para obtener los valores distintos
DimMetodoPago	Limpieza	Campo MetodoPagoKey = CASE [MetodoPago] WHEN "Efectivo" THEN 1 WHEN "Tarjeta credito" THEN 2 WHEN "Tarjeta debito" THEN 3 WHEN "Transferencia" THEN 4 WHEN "Billetera digital" THEN 5 ELSE 0 END
DimMetodoPago	Limpieza	Quita las columnas Pagos
Salida DimMetodoPago	Salida	Escribe DimMetodoPago.csv
Filtrar ordenes completadas	Limpieza	Filtro: [EstadoOrden] = "Completada"
Filtrar ordenes completadas	Limpieza	Campo FechaKey = YEAR([FechaOrden]) * 10000 + MONTH([FechaOrden]) * 100 + DAY([FechaOrden])
Filtrar ordenes completadas	Limpieza	Campo ClienteKey = IFNULL([ClienteID], -1)

Paso	Tipo	Operación
Filtrar ordenes completadas	Limpieza	Campo PromocionKey = IFNULL([PromocionID], -1)
Filtrar ordenes completadas	Limpieza	Campo CanalKey = CASE [CanalVenta] WHEN "Tienda" THEN 1 WHEN "Web" THEN 2 WHEN "App" THEN 3 ELSE 0 END
Filtrar ordenes completadas	Limpieza	Campo TiendaKey = [TiendaID]
Filtrar ordenes completadas	Limpieza	Quita las columnas ClienteID, PromocionID, TiendaID, EstadoOrden, CanalVenta
Detalle + Orden	Unión	Unión interna por [OrdenID] = [OrdenID]
Ventas + Producto	Unión	Unión interna por [ProductoID] = [ProductoID]
FactVentas	Limpieza	Campo VentaKey = [DetalleOrdenID]
FactVentas	Limpieza	Campo CantidadVendida = [Cantidad]
FactVentas	Limpieza	Campo MontoBruto = ROUND([Cantidad] * [PrecioUnitario], 2)
FactVentas	Limpieza	Campo MontoNeto = ROUND([MontoBruto] - [DescuentoMonto], 2)
FactVentas	Limpieza	Campo CostoTotal = ROUND([Cantidad] * [CostoUnitario], 2)
FactVentas	Limpieza	Campo MargenBruto = ROUND([MontoNeto] - [CostoTotal], 2)
FactVentas	Limpieza	Quita las columnas DetalleOrdenID, ProductoID, Cantidad, SKU, NombreProducto, Marca, Categoria, Proveedor, PaisProveedor, CostoUnitario, PrecioLista, RangoPrecio, FechaLanzamiento, Activo, FechaOrden
Salida FactVentas	Salida	Escribe FactVentas.csv
Devolucion + Detalle	Unión	Unión interna por [DetalleOrdenID] = [DetalleOrdenID]
FactDevoluciones	Limpieza	Campo DevolucionKey = [DevolucionID]
FactDevoluciones	Limpieza	Campo FechaKeyDevolucion = YEAR([FechaDevolucion]) * 10000 + MONTH([FechaDevolucion]) * 100 + DAY([FechaDevolucion])
FactDevoluciones	Limpieza	Campo MotivoKey = CASE [Motivo] WHEN "Producto defectuoso" THEN 1 WHEN "Producto equivocado" THEN 2 WHEN "Cambio de opinion" THEN 3 WHEN "Danado en transporte" THEN 4 WHEN "Talla o presentacion" THEN 5 ELSE 0 END
FactDevoluciones	Limpieza	Quita las columnas DevolucionID, ProductoID, FechaDevolucion, Cantidad, PrecioUnitario, DescuentoMonto, FechaKey, PromocionKey, CanalKey, FechaOrden, OrdenID, SKU, NombreProducto, Marca, Categoria, Proveedor, PaisProveedor, CostoUnitario, PrecioLista, RangoPrecio, FechaLanzamiento, Activo, Motivo
FactDevoluciones	Limpieza	Renombra FechaKeyDevolucion a FechaKey
Salida FactDevoluciones	Salida	Escribe FactDevoluciones.csv
Pago + Orden	Unión	Unión interna por [OrdenID] = [OrdenID]
FactPagos	Limpieza	Campo PagoKey = [PagoID]
FactPagos	Limpieza	Campo FechaKeyPago = YEAR([FechaPago]) * 10000 + MONTH([FechaPago]) * 100 + DAY([FechaPago])
FactPagos	Limpieza	Campo MetodoPagoKey = CASE [MetodoPago] WHEN "Efectivo" THEN 1 WHEN "Tarjeta credito" THEN 2 WHEN "Tarjeta debito" THEN 3 WHEN "Transferencia" THEN 4 WHEN "Billetera digital" THEN 5 ELSE 0 END

Paso	Tipo	Operación
FactPagos	Limpieza	Campo MontoPagado = [Monto]
FactPagos	Limpieza	Quita las columnas PagoID, Monto, FechaPago, MetodoPago, FechaKey, PromocionKey, FechaOrden
FactPagos	Limpieza	Renombra FechaKeyPago a FechaKey
Salida FactPagos	Salida	Escribe FactPagos.csv

Anexo D. Consultas y campos calculados del dashboard

Consultas de Consultas_Dashboard.sql. Se ejecutan sobre DW_Retail y su resultado forma las dos tablas del extracto que usa el libro de Tableau.

```
/*
=====
ICC-321 Inteligencia de Negocios, Práctica 1
Actividad 7: consultas que alimentan el dashboard

Estudiantes: Elías De La Cruz (ID 10155971), Enger Douglas (ID 10144675)
Base: DW_Retail (no se consulta la base operacional)

Cada bloque "-- @tabla" produce una tabla del extracto de Tableau
(Dashboard_DW_Retail.hyper). Las consultas solo unen hechos con sus
dimensiones y agregan; ninguna medida se recalcula desde el origen.
Las etiquetas de categoría y promoción se escriben con tilde para que
se lean bien en el tablero.
=====
*/

USE DW_Retail;

-- @tabla Ventas
/* Grano: una línea de orden completada (el mismo de FactVentas). */
SELECT
    f.VentaKey,
    f.OrdenID,
    d.Fecha,
    d.Año,
    d.Trimestre,
    d.Mes,
    d.NombreMes,
    CASE p.Categoria
        WHEN 'Bebes' THEN N'Bebés'
        WHEN 'Electrodomesticos' THEN N'Electrodomésticos'
        WHEN 'Papeleria' THEN N'Papelería'
        WHEN 'Tecnologia' THEN N'Tecnología'
        ELSE p.Categoria
    END AS Categoria,
    p.Marca,
    p.Proveedor,
    p.PaisProveedor,
    p.NombreProducto,
    p.RangoPrecio,
    c.Segmento,
    c.Provincia AS ProvinciaCliente,
    t.NombreTienda,
    t.TipoTienda,
    t.Provincia AS ProvinciaTienda,
    ca.CanalVenta,
    REPLACE(REPLACE(pr.NombrePromocion, 'Valentin', N'Valentín'),
        'Sin promocion', N'Sin promoción') AS Promocion,
    CASE WHEN f.PromocionKey = -1 THEN N'Sin promoción'
        ELSE N'Con promoción' END AS TipoVenta,
    /* La campaña agrupa las ediciones de cada año de una misma promoción */
    CASE WHEN f.PromocionKey = -1 THEN N'Sin promoción'
        ELSE REPLACE(LEFT(pr.NombrePromocion, LEN(pr.NombrePromocion) - 5), 'Valentin', N'Valentín')
        + N' (' + CAST(CAST(pr.PorcentajeDto AS int) AS nvarchar(3)) + N' %)'
    END AS Campana,
    pr.PorcentajeDto,
    f.CantidadVendida,
    f.MontoBruto,
    f.DescuentoMonto,
    f.MontoNeto,
    f.CostoTotal,
    f.MargenBruto
FROM dbo.FactVentas f
JOIN dbo.DimFecha d ON d.FechaKey = f.FechaKey
JOIN dbo.DimProducto p ON p.ProductoKey = f.ProductoKey
JOIN dbo.DimCliente c ON c.ClienteKey = f.ClienteKey
JOIN dbo.DimTienda t ON t.TiendaKey = f.TiendaKey
JOIN dbo.DimPromocion pr ON pr.PromocionKey = f.PromocionKey
JOIN dbo.DimCanal ca ON ca.CanalKey = f.CanalKey;

-- @tabla RentabilidadProducto
/* Consulta entre hechos (drill-across): FactVentas y FactDevoluciones se
agregan por separado al mismo nivel, año y producto, y se unen por sus
dimensiones conformadas. Así el reembolso se resta del margen sin
duplicar ninguna de las dos medidas. Las ventas se ubican en el año de
la venta y las devoluciones en el año de la devolución. */
WITH v AS (
    SELECT d.Año, f.ProductoKey,
```

```

SUM(f.MontoNeto) AS MontoNeto,
SUM(f.CostoTotal) AS CostoTotal,
SUM(f.MargenBruto) AS MargenBruto,
COUNT(*) AS LineasVendidas,
SUM(f.CantidadVendida) AS UnidadesVendidas
FROM dbo.FactVentas f
JOIN dbo.DimFecha d ON d.FechaKey = f.FechaKey
GROUP BY d.Anio, f.ProductoKey
),
dv AS (
SELECT d.Anio, f.ProductoKey,
COUNT(*) AS Devoluciones,
SUM(f.CantidadDevuelta) AS UnidadesDevueltas,
SUM(f.MontoReembolso) AS MontoReembolso
FROM dbo.FactDevoluciones f
JOIN dbo.DimFecha d ON d.FechaKey = f.FechaKey
GROUP BY d.Anio, f.ProductoKey
)
SELECT
COALESCE(v.Anio, dv.Anio) AS Anio,
CASE p.Categoria
WHEN 'Bebes' THEN N'Bebés'
WHEN 'Electrodomesticos' THEN N'Electrodomésticos'
WHEN 'Papeleria' THEN N'Papelería'
WHEN 'Tecnologia' THEN N'Tecnología'
ELSE p.Categoria
END AS Categoria,
p.Marca,
p.Proveedor,
p.PaisProveedor,
p.NombreProducto,
ISNULL(v.MontoNeto, 0) AS MontoNeto,
ISNULL(v.CostoTotal, 0) AS CostoTotal,
ISNULL(v.MargenBruto, 0) AS MargenBruto,
ISNULL(v.LineasVendidas, 0) AS LineasVendidas,
ISNULL(v.UnidadesVendidas, 0) AS UnidadesVendidas,
ISNULL(dv.Devoluciones, 0) AS Devoluciones,
ISNULL(dv.UnidadesDevueltas, 0) AS UnidadesDevueltas,
ISNULL(dv.MontoReembolso, 0) AS MontoReembolso
FROM v
FULL JOIN dv ON dv.Anio = v.Anio AND dv.ProductoKey = v.ProductoKey
JOIN dbo.DimProducto p ON p.ProductoKey = COALESCE(v.ProductoKey, dv.ProductoKey);

```

Campos calculados definidos en el libro de Tableau:

Tabla 15. Campos calculados del dashboard.

Tabla del extracto	Campo	Fórmula
Ventas	Margen %	SUM([MargenBruto]) / SUM([MontoNeto])
Ventas	Órdenes	COUNTD([OrdenID])
Ventas	Ticket promedio	SUM([MontoNeto]) / COUNTD([OrdenID])
Ventas	Venta neta	SUM([MontoNeto]) / 1000000
Ventas	Margen bruto	SUM([MargenBruto]) / 1000000
Ventas	Participación en la venta	SUM([MontoNeto]) / TOTAL(SUM([MontoNeto]))
Ventas	Filtro del tablero	([Parameters].[p_anio] = "Todos" OR STR([Anio]) = [Parameters].[p_anio]) AND ([Parameters].[p_categoria] = "Todas" OR [Categoria] = [Parameters].[p_categoria]) AND ([Parameters].[p_canal] = "Todos" OR [CanalVenta] = [Parameters].[p_canal])
RentabilidadProducto	Tasa de devolución	SUM([Devoluciones]) / SUM([LineasVendidas])
RentabilidadProducto	Margen %	SUM([MargenBruto]) / SUM([MontoNeto])
RentabilidadProducto	Margen después de devoluciones	SUM([MargenBruto]) - SUM([MontoReembolso])

Tabla del extracto	Campo	Fórmula
RentabilidadProducto	Margen después de devoluciones	$(SUM([MargenBruto]) - SUM([MontoReembolso])) / 1000000$
RentabilidadProducto	Margen bruto	$SUM([MargenBruto]) / 1000000$
RentabilidadProducto	Reembolsos	$SUM([MontoReembolso]) / 1000000$
RentabilidadProducto	Margen que se va en reembolsos	$SUM([MontoReembolso]) / SUM([MargenBruto])$
RentabilidadProducto	Filtro del tablero	$([Parameters].[p_anio] = "Todos" OR STR([Anio]) = [Parameters].[p_anio]) AND ([Parameters].[p_categoria] = "Todas" OR [Categoria] = [Parameters].[p_categoria])$

Parámetros usados como filtros en los tableros:

Tabla 16. Parámetros de los tableros.

Parámetro	Valores
Año	Todos, 2023, 2024, 2025, 2026
Categoría	Todas, Alimentos y bebidas, Bebés, Cuidado personal, Deportes, Electrodomésticos, Hogar y limpieza, Mascotas, Moda, Papelería, Tecnología
Canal	Todos, Tienda, Web, App